

Architecture

1. System overview

The platform orchestrates GPU inference infrastructure end-to-end:

1. Acquire/track servers from provider APIs (Clore.ai first)
2. Configure and deploy runtime (vLLM and model assets) through SSH + Ansible
3. Establish model reachability from a stable control plane
4. Route requests through LiteLLM Proxy using OpenAI-compatible APIs
5. Capture structured outcomes, diagnostics, and artifacts for future decision-making

2. Logical components

Frontend (Next.js)

- Operator-facing dashboard
- Server inventory and deployment status
- Log browsing and artifact download
- Future: tunnel health and cost views

Backend API (FastAPI)

- Public control-plane API
- Validates requests and persists state
- Triggers long-running jobs through Celery
- Exposes filtered data for UI and automation

Worker (Celery)

- Provisioning orchestration
- SSH checks and remote command execution
- Ansible playbook execution pipeline
- Tunnel lifecycle and health checks

PostgreSQL

- Source of truth for infrastructure state and historical outcomes
- Stores run-level details that power future intelligence/recommendation features

Redis

- Queue broker and result backend for async workflows

LiteLLM Proxy

- Stable OpenAI-compatible API endpoint
- Maps logical model names to active remote endpoints
- Future: aliasing, weighted routing, fallback, QoS

Ansible assets (Git-first)

- Playbooks organized for collaboration and review

- Metadata tags for compatibility and reproducibility

3. Interaction flow

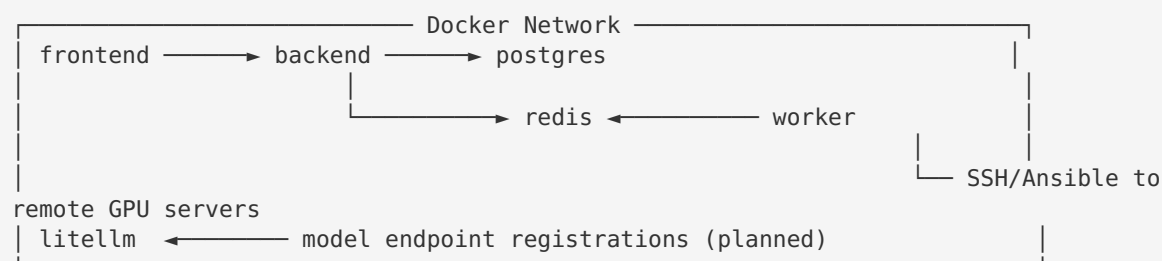


4. Data model strategy

The initial schema is intentionally normalized and extensible.

- `users` : owner/operator identity (single-user now, multi-user later)
- `provider_accounts` : external provider credentials/metadata reference
- `servers` : rented machine inventory and capabilities
- `playbooks` : automation assets and source-control metadata
- `model_deployments` : runtime mapping of model to server + tunnel info
- `api_keys` : internal/external key references and rotation metadata
- `task_runs` : async execution records (state, logs path, timing)
- `audit_logs` : immutable event stream for traceability

5. Deployment topology (Docker Compose)



6. Logging and knowledge-base foundation

The platform treats logs as first-class data:

- command/playbook outputs

- error traces
- execution metadata (server GPU/CUDA/VRAM, playbook commit, model details)

This enables future intelligence features:

- success-rate scoring
- compatibility suggestions
- failure similarity analysis
- eventual semantic search

7. Security baseline (v1)

- Single-user by default, no public auth gateway yet
- Secret material via `.env` and container env vars
- Avoid storing sensitive SSH private keys in DB for v1 unless encrypted workflow is added
- API keys tracked with metadata and revocation flags

8. Open extension points

- Multi-provider adapter system beyond Clore.ai
- Dynamic LiteLLM route management API integration
- Cost tracking and budget controls
- Retry policies and fallback deployment strategies
- Websocket/streamed run output in UI